

From Cognitive Maps to Neural Architectures

A Gradient-Based, End-to-End Differentiable CSCG Pipeline

Arash Nikzad

Supervisor: Dr. Tristan Stober

Roadmap

I. Why CSCGs?

Cognitive maps and the hippocampal connection

II. Gradient-Based CSCG

From EM to differentiable forward in log-space

III. Adding Vision (VQ-VAE)

A soft-emission bridge to perception

IV. Joint Training

Phased schedule, loss balancing, stability

V. Experiments

MNIST gridworld and topology recovery

VI. What's Next

Dynamic clones, RNN-CSCG, exponential state machines

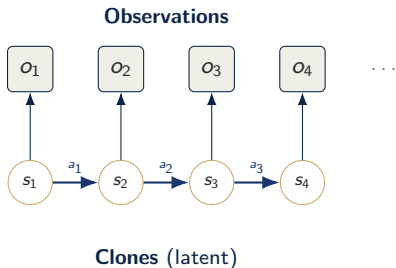


PART 1

Why CSCGs?

Cognitive maps from streams of experience

- An agent receives (o_t, a_t) from interaction with the world.
- It must infer a **structured internal model** of the environment — a *cognitive map*.
- **Clone-Structured Cognitive Graphs (CSCGs)** are an action-augmented *cloned HMM* that have been shown to recover environment topology under perceptual aliasing.
- Each observation symbol owns several latent *clones*; context disambiguates aliased percepts.



Why focus on CSCGs in particular?

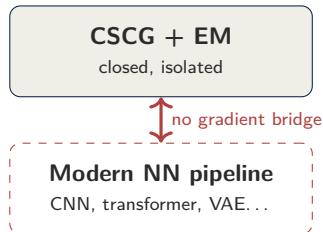
A neuroscientific anchor

“Learning produces an orthogonalized state machine in the hippocampus” (Sun et al., 2023; *Nature*) showed that the **activations of a CSCG model are the closest match to hippocampal recordings**, among the model classes tested.

- This suggests CSCGs carry a **useful inductive bias** for building cognitive maps.
- If the bias is biologically aligned, it is worth scaling — but today’s CSCGs are not built for scale.
- **Working hypothesis:** make the inductive bias of CSCGs *trainable inside modern neural pipelines*.

The limitation: CSCGs sit on an old foundation

- CSCGs are an **HMM** — a probabilistic model from the 1960s.
- Trained with **Expectation-Maximization** (Baum–Welch):
 - **E-step**: forward–backward to get posteriors.
 - **M-step**: closed-form re-estimation of π , T , B .
- Closed-form M-step requires *conjugate* emission structure.
- No mechanism to **compose** a CSCG with a CNN, a VQ-VAE, or any other neural module.
- Discrete observation alphabet must be supplied *from outside*.





PART 2

Gradient-Based CSCG

Step 1 — a gradient-trained CSCG

Goal

Replace EM with stochastic gradient descent so the CSCG becomes a **drop-in differentiable module**.

- Express the forward algorithm as a **log-space computational graph**.
- Optimize the negative log-likelihood directly with Adam.
- Use `logsumexp` for numerical stability over long episodes.
- Result: an action-conditioned CSCG that is end-to-end **differentiable in its parameters** (π, Θ) .

This is the prerequisite for everything that follows — once trainable by gradients, the CSCG can sit inside any neural pipeline.

Math — the classical HMM forward

Setup. Hidden states $s_t \in \mathcal{S} = \{1, \dots, N\}$, observations $o_t \in \{1, \dots, K\}$, actions $a_t \in \{1, \dots, A\}$.

- Initial distribution: $\bar{\pi}_j = \text{softmax}(\pi)_j$.
- Transitions: $T_{a,i,j} = \text{softmax}_j(\Theta_{a,i,\cdot})$.
- Emissions (CSCG): **deterministic**, $B_{s,o} = \mathbb{1}[\omega(s) = o]$.

Forward recursion ($\alpha_t(j) = P(o_{1:t}, s_t = j \mid a_{1:t-1})$):

$$\begin{aligned}\alpha_1(j) &= \bar{\pi}_j B_{j,o_1} \\ \alpha_{t+1}(j) &= \left[\sum_i \alpha_t(i) T_{a_t,i,j} \right] B_{j,o_{t+1}}\end{aligned}$$

Likelihood: $P(o_{1:T} \mid a_{1:T-1}) = \sum_j \alpha_T(j)$.

Two problems with raw α 's

1. **Numerical underflow** on long T .
2. Classical EM re-estimates parameters in closed form — *not* via gradients.

From EM to gradient descent: the log-space forward

Idea. Work in log-space, replace sums by logsumexp, treat the whole recursion as a TensorFlow graph; gradients flow into (π, Θ) via backprop.

Let $\text{logsumexp}_i u_i = \log \sum_i e^{u_i}$. The log-forward messages are

$$\begin{aligned}\log \alpha_1(j) &= \log \bar{\pi}_j + \log B_{j,o_1}, \\ \log \alpha_{t+1}(j) &= \text{logsumexp}_i [\log \alpha_t(i) + \log T_{a_t,i,j}] + \log B_{j,o_{t+1}}.\end{aligned}$$

Episode log-likelihood and NLL: $\ell = \text{logsumexp}_j \log \alpha_T(j)$, $\mathcal{L}_{\text{HMM}} = -\frac{1}{|B|} \sum \ell$.

Parameter update (gradient):

$$(\pi, \Theta) \leftarrow (\pi, \Theta) - \eta \nabla \mathcal{L}_{\text{HMM}}.$$

Why this is enough:

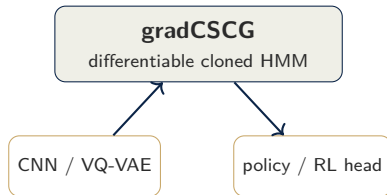
- Softmax keeps $\bar{\pi}$, T normalized.
- logsumexp is smooth, exact, stable.
- No conjugacy assumption; composable with any upstream module.

gradCSCG — summary of the first step

- Same model class as the classical CSCG, same clone semantics.
- Trained by SGD/Adam on the log-space forward NLL.
- Masked, batched recursion over padded minibatches.
- Viterbi decoding stays available for evaluation.

Why it matters

gradCSCG is the *interface* between the cognitive-map inductive bias and the rest of deep learning.





PART 3

Adding Vision via VQ-VAE

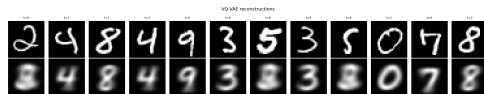
From integer observations to images

Today's CSCG observation:

- a single integer $o_t \in \{1, \dots, K\}$.
- Assumes someone else has discretized the world for us.

What a real agent sees:

- pixels — high-dimensional, noisy, aliased.
- the same place can look different each visit.

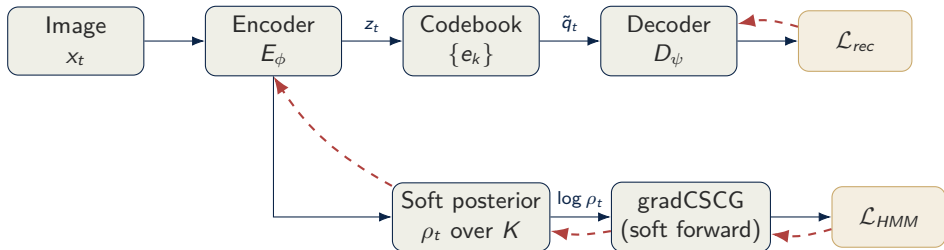


Top: input images. Bottom: VQ-VAE reconstructions and assigned token id.

Plan

Give the CSCG **eyes**: a VQ-VAE maps each image to a learned discrete token; gradCSCG consumes those tokens.

The neuralCSCG pipeline



Solid: forward pass. Dashed red: gradient flow. Codebook updated by **EMA**, not by gradients.

The eye — VQ-VAE: quantization & straight-through

Encoder + quantizer. Convnet $E_\phi : x \mapsto z \in \mathbb{R}^D$; codebook $\{e_k\}_{k=1}^K \subset \mathbb{R}^D$.

$$k_t = \arg \min_k \|z_t - e_k\|_2^2, \quad q_t = e_{k_t}.$$

Gradients cross the non-differentiable arg min by the **straight-through estimator**:

$$\tilde{q}_t = z_t + \text{sg}(q_t - z_t), \quad \frac{\partial \tilde{q}_t}{\partial z_t} = I.$$

Losses. $\mathcal{L}_{\text{rec}} = \frac{1}{|\mathcal{B}|} \sum_t \|x_t - \hat{x}_t\|_2^2$, $\mathcal{L}_{\text{commit}} = \frac{1}{|\mathcal{B}|} \sum_t \|\text{sg}(q_t) - z_t\|_2^2$.

Codebook (no gradient). EMA updates per code k with decay γ :

$$n_k \leftarrow \gamma n_k + (1 - \gamma) \sum_t \mathbb{I}[k_t = k], \quad m_k \leftarrow \gamma m_k + (1 - \gamma) \sum_t \mathbb{I}[k_t = k] z_t, \quad e_k \leftarrow m_k / \hat{n}_k.$$

The bridge — soft codebook posterior

We need the HMM likelihood to be **differentiable in** z_t so that gradients reach the encoder. A hard arg min is not.

Temperature-controlled soft posterior over the codebook

$$\log \rho_t(k) = \log \text{softmax}_k \left(-\|z_t - e_k\|_2^2 / \tau \right).$$

Two key properties:

- **Differentiable in** z_t (so also in ϕ).
- **Consistency:** as $\tau \rightarrow 0$, ρ_t concentrates on k_t and reduces to the hard one-hot assignment.

ρ_t is the soft handshake between perception and sequence learning.

Soft-emission forward pass — the key derivation

Recall: state j deterministically emits token $\omega(j)$. In the hard forward, the emission term is $\log B_{j,o_t}$ (i.e. $-\infty$ unless $\omega(j) = o_t$).

Replace the hard emission with the soft log-posterior of $\omega(j)$:

$$\begin{aligned}\log \alpha_1^s(j) &= \log \bar{\pi}_j + \log \rho_1(\omega(j)) \\ \log \alpha_{t+1}^s(j) &= \log \text{sumexp}_i [\log \alpha_t^s(i) + \log T_{a_t,i,j}] + \log \rho_{t+1}(\omega(j))\end{aligned}$$

Soft log-likelihood: $\ell^s = \log \text{sumexp}_j \log \alpha_T^s(j)$.

Consistency (proof sketch).

Why it matters.

If $\rho_t = \text{onehot}(o_t)$, then $\log \rho_t(\omega(j)) = \log B_{j,o_t}$. Gradients propagate $\ell^s \rightarrow \rho \rightarrow z \rightarrow \phi$: the so the soft forward reduces *exactly* to the hard topological objective now *shapes perception*. forward as $\tau \rightarrow 0$.



PART 4

Joint Training

The joint objective

Combined loss at joint training step t

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{rec}} + \beta \mathcal{L}_{\text{commit}} + \lambda_t \tilde{\mathcal{L}}_{\text{HMM}} + \alpha_{\text{div}} \mathcal{L}_{\text{div}}$$

- $\mathcal{L}_{\text{rec}}, \mathcal{L}_{\text{commit}}$ — the VQ-VAE terms (perception stays faithful).
- $\tilde{\mathcal{L}}_{\text{HMM}}$ — the *length-normalized* soft NLL (topology pressure).
- $\mathcal{L}_{\text{div}}$ — diversity penalty (anti-collapse).
- λ_t — annealed sequence-loss weight.

Naive joint training collapses — the next slide is why, and how we fix it.

Loss balancing — the four knobs

1. Length normalization

The raw NLL grows $O(T)$, dwarfing the $O(1)$ reconstruction term.

$$\tilde{\mathcal{L}}_{\text{HMM}} = -\frac{1}{|\mathcal{B}|} \sum \frac{1}{T} \ell^s.$$

2. Weight annealing

Let the codebook stabilize under reconstruction before topology pressure switches on.

$$\lambda_t = \lambda \cdot \min\left(1, t/T_{\text{anneal}}\right).$$

3. Diversity penalty

Let $\bar{\rho}(k) = \frac{1}{|\mathcal{B}|} \sum_t \rho_t(k)$.

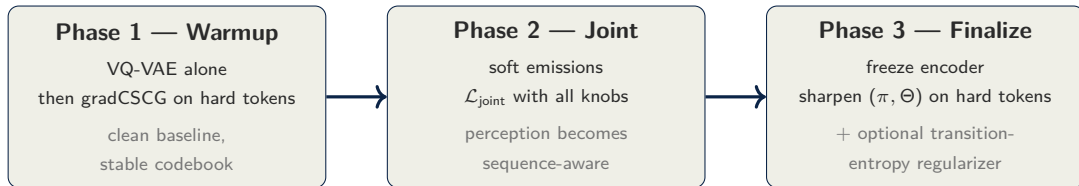
$$\mathcal{L}_{\text{div}} = \log K - H(\bar{\rho}) \geq 0.$$

Vanishes only at uniform usage — counters codebook collapse.

4. Anti-collapse safeguards

- Perplexity-monitored best-checkpoint snapshot.
- Optional λ -throttling and rollback.
- Dead-code revival from encoder cluster centroids.

Three-phase training schedule



- Phase 1 gives a strong baseline and prevents Phase 2 from starting in a bad basin.
- Phase 2 is where soft emissions actually couple the two modules.
- Phase 3 produces the final, sharp transition matrix used at evaluation.

One joint training step — the algorithm

Per-batch update

1. **Encode:** $z_t \leftarrow E_\phi(x_t)$.
2. **Quantize:** $q_t, \tilde{q}_t \leftarrow \text{quantize}(z_t)$; update codebook by EMA.
3. **Reconstruct:** $\hat{x}_t \leftarrow D_\psi(\tilde{q}_t)$; compute $\mathcal{L}_{\text{rec}}, \mathcal{L}_{\text{commit}}$.
4. **Soft posterior:** $\log \rho_t \leftarrow \log \text{softmax}(-\|z_t - e.\|^2/\tau)$.
5. **Soft forward:** run log-space recursion, get ℓ^s .
6. **Balance:** $\tilde{\mathcal{L}}_{\text{HMM}} \leftarrow -\ell^s/T$; $\mathcal{L}_{\text{div}} \leftarrow \log K - H(\bar{\rho})$.
7. **Joint loss:** $\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{rec}} + \beta \mathcal{L}_{\text{commit}} + \lambda_t \tilde{\mathcal{L}}_{\text{HMM}} + \alpha_{\text{div}} \mathcal{L}_{\text{div}}$.
8. **Update** $(\phi, \psi, \pi, \Theta)$ with Adam on $\nabla \mathcal{L}_{\text{joint}}$.

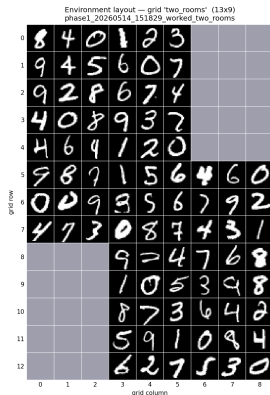


PART 5

Experiments & Results

Does it work? — the MNIST gridworld benchmark

- 2-D grid; each cell carries a **digit class**.
- *Visiting a cell returns a random MNIST sample of that digit.*
⇒ same place never looks identical twice.
- Cells sharing a digit are **perceptually aliased**.
- Four actions (up/down/left/right) with sticky walls.
- Ground-truth topology is exactly known ⇒ controlled testbed for **topology recovery**.



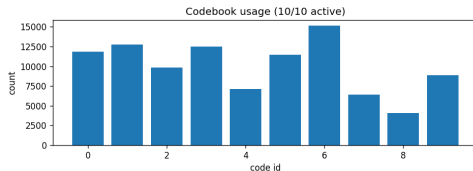
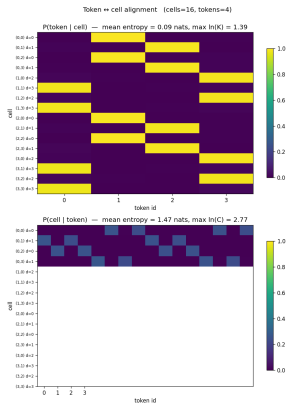
two_rooms (13 × 9): two offset rooms; shared digits
cause aliasing.

Five benchmark environments

Environment	Grid	Tokens K	Clone states N	Character
aliased	4×4	4	21	4 digits, each $\times 4$ — controlled aliasing walls + repeats along corridors
corridors	5×5	6	31	
room (uniform)	6×6	10	201	rectangular room, uniform clones
room (dynamic)	6×6	10	57	dynamic clone allocation
two_rooms	13×9	10	151	two interconnected rooms (hardest)

- Trajectories: action-conditioned random walks — 4–10 episodes of 10,000 steps.
- Optional coverage-biased rollout for rare (cell, action) pairs.
- All five environments run through the full Phase 2 $\rightarrow$ finalization pipeline.

Results I — perception is clean



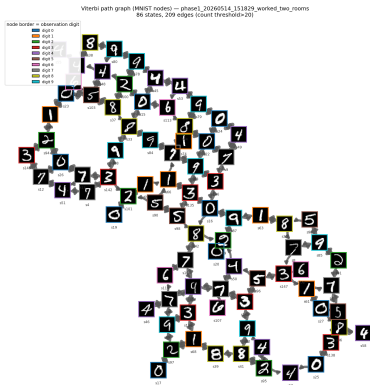
Codebook utilization on `two_rooms` — all codes active.

- Mean tokens per cell: **1.0–1.2**.
- $H(\text{token} | \text{place}) = 0.09\text{--}0.23$ nats.
- Perplexity $\approx$ number of digit classes; no collapse.

Token↔place alignment (aliased). Top: $P(\text{token} | \text{place})$.

Bottom: $P(\text{place} | \text{token})$.

Results II — clones absorb the aliasing



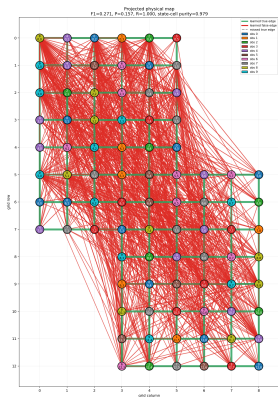
Decoded clone-state graph for two_rooms: each node is an MNIST sample of the digit it represents.

- Clones map cleanly to physical places.
- Visit-weighted **state-to-place purity** = **0.95–0.98**.
- Every visited place is represented by at least one clone.
- Decoded clone graph already exposes the recovered topology.

Take-away

The CSCG inductive bias survives the move to gradient training *and* the move to learned visual tokens.

Results III — topology is recovered



two_rooms learned map projected on the true grid. Green: correct edges. Red: false. Grey dashed: missed (none).

- Map-edge recall = 1.00 in every scored run.
- The learned graph *contains* the entire true map.
- Precision rises sharply with the edge threshold.

η	0.01	0.05	0.10	0.30
Precision	0.24	0.29	0.39	0.77
Recall	1.00	1.00	1.00	1.00
F1	0.39	0.45	0.56	0.87

Edge-threshold sweep on corridors.

Results IV — cross-environment summary

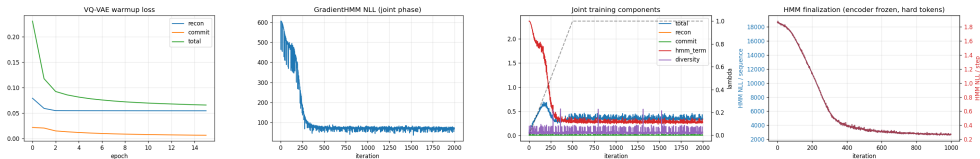
Environment	Grid	K	N	Perplexity	Clone pur.	State→place	Action acc.	Map recall	Best F1
aliased	4×4	4	21	4.00	0.84	—	—	—	—
corridors	5×5	6	31	5.71	0.86	0.98	1.00	1.00	0.87
room (uniform)	6×6	10	201	6.63	0.90	0.98	0.94	1.00	0.67
room (dynamic)	6×6	10	57	6.56	0.83	0.95	0.97	1.00	0.85
two_rooms	13×9	10	151	9.46	0.88	0.98	0.99	1.00	0.86

Headline

100% map-edge recall on every environment with the metric suite; clone-state purity **0.95–0.98**; action-outcome accuracy **0.94–1.00**.

- Scales smoothly from 4×4 to 13×9 .
- Dynamic clones get most of the F1 with $4 \times$ fewer states.
- Open frontier: **precision** at small thresholds.

Training dynamics — the phases work as intended



two_rooms: VQ-VAE warmup, joint-phase components (reconstruction, commitment, length-normalized HMM, diversity), and the pure-HMM finalization.

- Reconstruction stays **stable** through joint training $\Rightarrow$ no collapse.
- Length-normalized HMM term **decreases steadily** once λ_t ramps in.
- Finalization (encoder frozen) drives a final sharp drop in HMM NLL.



PART 6

Limitations & Outlook

What still doesn't work as well as it should

Perception bottleneck

- Each observation collapses to *one* integer token.
- Severely limits how much an agent can “see”.
- Codebook size is a hard scalability cap.
- **Dynamic clone allocation** helps but is still primitive.

Edge precision

- Recall is perfect; precision is threshold-dependent.
- Many *weak* false edges remain in the learned graph.
- Needs sharper transition regularization.

Scaling and training cost

- Forward recursion is sequential in time $\Rightarrow$ slow.
- Transition tensor is $O(A \cdot N^2)$ in memory.
- Joint training is delicate; every knob matters.
- Learning is slower than it should be.

Scope

- Validated on controlled MNIST gridworlds.
- Not yet a 3-D image-based environment.

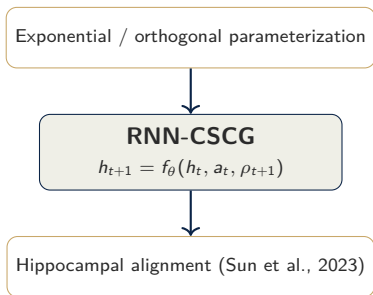
Maybe another way — RNN-CSCG and exponential RNNs

- Replace the cloned HMM with a **recurrent neural network** constrained to preserve cloning semantics.
- Hidden state h_t evolves as

$$h_{t+1} = f_{\theta}(h_t, a_t, \rho_{t+1}),$$

with f_{θ} chosen to keep transitions sparse and orthogonal.

- **Exponential / orthogonal RNNs** could realize the "orthogonalized state machine" highlighted by the hippocampal study — the very property that linked CSCGs to the brain in the first place.



Same inductive bias, more scalable substrate.

Take-aways

1. CSCGs carry a useful, neuroscientifically anchored inductive bias for cognitive maps — but classical EM training isolates them from modern deep learning.
2. A **log-space, differentiable forward pass** turns the CSCG into a first-class neural module (**gradCSCG**).
3. Coupling gradCSCG to a **VQ-VAE** via a soft codebook posterior gives the first end-to-end pipeline from *pixels to cognitive map*.
4. Joint training is made stable by principled loss balancing (length normalization, λ -annealing, diversity penalty).
5. On MNIST gridworlds we recover topology with **100% edge recall** and **state-to-place purity 0.95–0.98**.
6. The frontier: *richer perception, sharper transitions*, and a **scalable RNN-CSCG** successor.

Thank you.

Questions & discussion welcome.

Arash Nikzad · Supervisor: Dr. Tristan Stober

neuralCSCG | preprint and code in the project repository